

The Jupiter Developer Platform is live. Previous portal users keep their rate limits for free until 30 June 2026 — set up billing on the [new platform](#) before then. See the [Migration Guide](#) for details.



☰ Add Fees To Swap

Add Fees To Swap

📄 Copy page ▼

Add integrator fees to Metis swaps and collect revenue from user trades.

⚠️ **Metis Swap API** is no longer actively maintained and has been superseded by [Swap V2](#).

Add a platform fee to Metis swaps by passing `platformFeeBps` (in basis points) when requesting a quote and `feeAccount` (any valid token account for the swap pair mint) when building the swap transaction. No referral program setup is required for the Metis Swap API.

📘 INFO

As of January 2025, when integrating the Metis Swap API, you no longer need to use the Referral Program to set up a `referralAccount` and `referralTokenAccount` to collect fees from the swaps you provide to the end users.

Simply, just pass in any valid token account as the `feeAccount` parameter in the Metis Swap API.

However, do note that **it is still applicable to the Trigger API**.

📌 NOTE

You can still find information about the Referral Program.

The Referral Program is an open source program by Jupiter to provide referral fees for integrators who are integrating Jupiter Swap and Jupiter Limit Order. You can check out the code [here](#) to gain a better understanding of how it works.

The Jupiter Developer Platform is live. Previous portal users keep their rate limits for free until 30 June 2026 — set up billing on the [new platform](#) before then. See the [Migration Guide](#) for details.

By default, there are **zero** protocol fees on Jupiter Swap. Integrators have the option to introduce a platform fee denoted in basis points, e.g. **20 bps** for **0.2%** of the token input or output.

Important Notes

- For ExactIn swaps, the `feeAccount` 's mint can be either the **input mint or output mint** of the swap pair, and not any other mints.
- For ExactOut swaps, the `feeAccount` 's mint can only be the **input mint** of the swap pair.
- Example, if you swap JUP to USDC, you cannot take fees in SOL, it has to be part of the swap pair.
- It supports SPL and Token2022 tokens.
 - [Refer to this update for support of Token2022 tokens.](#)
- Referral Program is no longer required for Metis Swap API.

1. Set up

You will need to complete the prerequisites and understanding of [Environment Setup](#) and [Get Quote and Swap](#) guide as this is reliant on the Metis Swap API.

2. Set your fee in Quote

Setting your fee is simple, just add `platformFeeBps` parameter to the `/quote` endpoint.

In this example, we set `platformFeeBps` to `20` which equates to 0.2%.

The Jupiter Developer Platform is live. Previous portal users keep their rate limits for free until 30 June 2026 — set up billing on the [new platform](#) before then. See the [Migration Guide](#) for details.

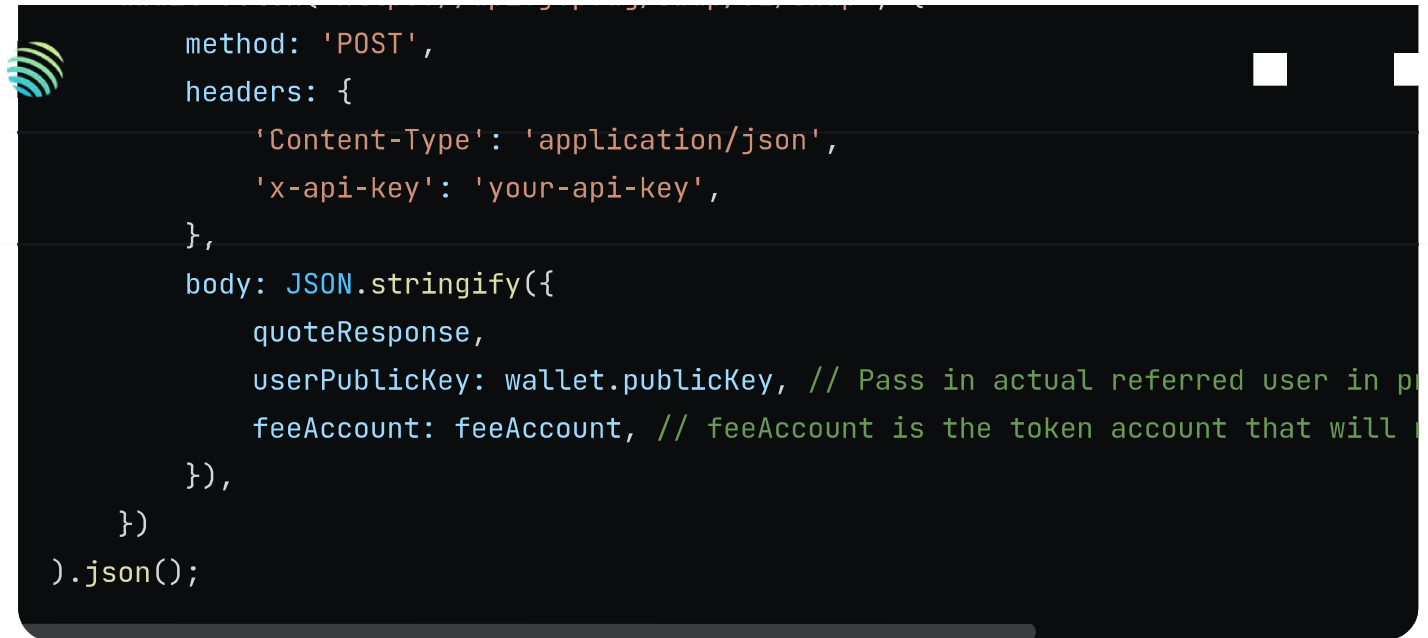
```
'https://api.jup.ag/swap/v1/quote?' +
'inputMint=So111111111111111111111111111111111112&' +
'outputMint=EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v&' +
'amount=100000&' +
'slippageBps=50&' +
'restrictIntermediateTokens=true&' +
'platformFeeBps=20&' +
'instructionVersion=V2' // required if you want to collect fees in Token2
),
{
  headers: {
    'x-api-key': 'your-api-key',
  },
}
).json();
```

3. Set your feeAccount in Swap

In the `/swap` endpoint, you will need to pass in the `feeAccount` parameter.

- The `feeAccount` is a token account that will receive the fees from the swap - the mint of the token account has to be part of the swap pair.
- Do ensure that the token account needs to be initialized beforehand and is the correct mint to receive the fees in.
- For ExactIn swaps, the `feeAccount` 's mint can be either the **input mint or output mint** of the swap pair, and not any other mints.
- For ExactOut swaps, the `feeAccount` 's mint can only be the **input mint** of the swap pair.
- Example, if you swap JUP to USDC, you cannot take fees in SOL, it has to be part of the swap pair.
- Refer to the **Create Token Account** section to create a token account.

The Jupiter Developer Platform is live. Previous portal users keep their rate limits for free until 30 June 2026 — set up billing on the [new platform](#) before then. See the [Migration Guide](#) for details.



4. Sign and send transaction

Finally, the user can sign the transaction and it can be submitted to the network to be executed. You can refer to the [Send Swap Transaction](#) guide to complete this step.

Create Token Account

To create a token account, you can use the following code or refer to [Solana Cookbook](#).

- The code creates the transaction to create the token account and handles the transaction signing and sending.
- If the token account already exists, it will not create and might throw an error such as `Provided owner is not allowed`.

The Jupiter Developer Platform is live. Previous portal users keep their rate limits for free until 30 June 2026 — set up billing on the [new platform](#) before then. See the [Migration Guide](#) for details.

```
const mintPubkey = new PublicKey(
  "JUPyiwrYJFskUPiHa7hkeR8VUtAeFoSYbKedZNSDvCN",
);

let ata = await createAssociatedTokenAccount(
  connection, // connection
  wallet, // fee payer
  mintPubkey, // mint
  wallet.publicKey, // owner of the token account
  // confirmOptions, // if you need to skip simulation and send the transaction
  // programId, // if you need to use a different token program id such as token
  // associatedTokenProgramId,
  // allowOwnerOffCurve, // if you need to allow the owner to be off curve
);
console.log(`ATA: ${ata.toBase58()}`);
```

For Trigger API Integrator Fee

Important Notes

- The Jupiter Swap project account for the Referral Program is `45ruCyfdRkWpRNGEqWzjCiXRHkZs8WXCLQ67Pnpye7Hp`.
- The `referralTokenAccount` can either be:
 - Input mint or the output mint on the swap for ExactIn.
 - Input mint ONLY on the swap for ExactOut.
- You can use the [Dashboard](#), [SDK](#) or [API](#) to set up the `referralAccount` and `referralTokenAccount` in this guide.

1. Set up

Obtain `referralAccount` and `referralTokenAccount`

There are 2 ways you can set up a referral account.

The Jupiter Developer Platform is live. Previous portal users keep their rate limits for free until 30 June 2026 — set up billing on the new platform before then. See the [Migration Guide](#) for details.



2. Use our SDK to create them. You can use the **example scripts** to create.

Obtain `mintAccount`

As for the mint account, assuming you have an interface where a user swaps, you will know up front what are the input or output mints. For the sake of example, we will use a hardcoded mint public key.

[illegible]

2. Get your referral token account for feeAccount

In order to refer and receive fees from all types of tokens, you will need to have already initialize `referralTokenAccounts` (owned by your `referralAccount`) for the mint in the order.

In this code block, we will be using the SDK to try to find the `referralTokenAccount` based on our previously defined `referralAccount` and `mintAccount`. If the token account is not found, it will send a transaction to the network to attempt to initialize one for the mint.

The Jupiter Developer Platform is live. Previous portal users keep their rate limits for free until 30 June 2026 — set up billing on the [new platform](#) before then. See the [Migration Guide](#) for details.

```
const { tx, referralTokenAccountPubKey } = await provider.initializeReferralTokenAccount({
  payerPubKey: wallet.publicKey,
  referralAccountPubKey: referralAccount,
  mint: mintAccount,
});

const referralTokenAccount = await connection.getAccountInfo(referralTokenAccountPubKey);

// Attempt to initialize a token account
if (!referralTokenAccount) {
  const signature = await sendAndConfirmTransaction(connection, tx, [wallet]);
  console.log({ signature, referralTokenAccountPubKey: referralTokenAccountPubKey });

  // Since initialized, it will carry on
} else {
  console.log(`referralTokenAccount ${referralTokenAccountPubKey.toBase58()} found`);
};

const feeAccount = referralTokenAccountPubKey;
console.log(feeAccount);
```

If you are confident that the `referralTokenAccount` for specific mints have been created, you can use this method to get it. Do note that, even if the token account is not initialized, it will still return a pubkey as it is a Program Derived Address and is deterministic. [Read more here.](#)

```
const [feeAccount] = PublicKey.findProgramAddressSync(
  [
    Buffer.from("referral_ata"), // A string that signifies the account type,
    referralAccount.toBuffer(), // The public key of the referral account controller,
    mintAccount.toBuffer(), // The mint public key, converted into a buffer.
  ],
  new PublicKey("REFER4ZgmyYx9c6He5XfaTMiGfdLwRnkV4RPp9t9iF3") // The public key of the referral program
);
```

The Jupiter Developer Platform is live. Previous portal users keep their rate limits for free until 30 June 2026 — set up billing on the [new platform](#) before then. See the [Migration Guide](#) for details.

Setting your referral fee is simple, just add `feeAccount` and `params.feeBps` parameters to the  `/createOrder` endpoint.

In this example, we set `params.feeBps` to `20` which equates to 0.2%.

```
const createOrderResponse = await (
  await fetch('https://api.jup.ag/trigger/v1/createOrder', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'x-api-key': 'your-api-key',
    },
    body: JSON.stringify({
      inputMint: inputMint.toString(),
      outputMint: outputMint.toString(),
      maker: "jdocuPgEAjMfihABsPgKEvYtsmMzjUHeq9LX4Hvs7f3",
      payer: "jdocuPgEAjMfihABsPgKEvYtsmMzjUHeq9LX4Hvs7f3",
      params: {
        makingAmount: "1000000",
        takingAmount: "300000",
        // slippageBps: "", // Optional, by nature, trigger orders execute
        // expiredAt: "", // In unix seconds (e.g. Date.now()/1_000) or optional
        feeBps: "20", // feeBps is the amount of fees in bps that will be
      },
      computeUnitPrice: "auto",
      feeAccount, // feeAccount is the token account that will receive the fee
      // wrapAndUnwrapSol: true, // Default true or optional
    })
  })
).json();
```


The Jupiter Developer Platform is live. Previous portal users keep their rate limits for free until 30 June 2026 — set up billing on the [new platform](#) before then. See the [Migration Guide](#) for details.

[Support](#)[Guidelines](#)[Legal](#)[Brand Kit](#)[Status](#)

Powered by **mintlify**

